

```

1  import os
2  import sys
3  import asyncio
4  from datetime import datetime
5
6  # for abstract class
7  from abc import abstractmethod
8
9  # LLM
10 import litellm
11
12 # internal agent CrewAI
13 from crewai import Agent, Crew, Process, Task, LLM
14 from crewai.events.event_bus import crewai_event_bus
15 from crewai.events.base_events import reset_emission_counter
16 from crewai.events.event_context import _event_id_stack, EventContextConfig,
17 _event_context_config
18 # use flow for blueprint diff analysis
19 from crewai.flow import Flow, listen, start
20
21 # Now Python can seamlessly see and import the centralized helper utility cleanly!
22 from sources.agents.agent_helper import (
23     write_file,
24     delete_file,
25     render_kwargs_prompt,
26     get_logger,
27     parse_args,
28     datetime_for_agent
29 )
30
31 # super agent
32 from sources.agents.subagent_super import AbstractSubAgent
33
34 # =====
35 # GLOBAL CONFIGURATION PATHS - CONFIG HERE TO CUSTOMIZE DIRECTORY STRUCTURE
36 # =====
37
38 # Crew Agent backstory prompts
39 PROMPT_TEMPLATE_BACKSTORY_SOLUTION_SENTINEL =
40 "agent_csro.backstory.prompt.solution-sentinel.md"
41 PROMPT_TEMPLATE_BACKSTORY_BA = "agent_csro.backstory.prompt.ba.md"
42 PROMPT_TEMPLATE_BACKSTORY_SA = "agent_csro.backstory.prompt.sa.md"
43 PROMPT_TEMPLATE_BACKSTORY_DIFF_ANALYZER =
44 "agent_csro.backstory.prompt.diff-blueprint-analyzer.md"
45
46 # Crew Agent Task `expected_output` prompts
47 PROMPT_TEMPLATE_EXPECTED_SOLUTION_SENTINEL = "agent_csro.expected.solution-sentinel.md"
48 PROMPT_TEMPLATE_EXPECTED_BA = "agent_csro.expected.ba.md"
49 PROMPT_TEMPLATE_EXPECTED_SA = "agent_csro.expected.sa.md"
50 PROMPT_TEMPLATE_EXPECTED_DIFF_ANALYZER =
51 "agent_csro.expected.diff-blueprint-analyzer.md"
52
53 # Crew Agent Task `task_description` prompts
54 PROMPT_TEMPLATE_TASK_SOLUTION_SENTINEL = "agent_csro.task.solution-sentinel.md"
55 PROMPT_TEMPLATE_TASK_BA = "agent_csro.task.ba.md"
56 PROMPT_TEMPLATE_TASK_SA = "agent_csro.task.sa.md"
57 PROMPT_TEMPLATE_TASK_DIFF_ANALYZER = "agent_csro.task.diff-blueprint-analyzer.md"
58
59 # CSRO Log files
60 CSRO_RAW_FILE = "chief-solution-review.md"
61 CSRO_LOG_FILE = "chief-solution-review_log.md"
62 CSRO_DA_FILE = "chief-solution-diff-analysis.md"
63 CSRO_LOG_DA_FILE = "chief-solution-diff-analysis_log.md"
64
65 DEFAULT_CSRO_LANGUAGE = "English"
66
67 # support for executing workflow
68 def __execute_function_until_complete__(func_pointer, **kwargs):
69     # use asyncio to run safely while CI/CD doesn't have loop under background

```

```

66     try:
67         loop = asyncio.get_event_loop()
68     except RuntimeError:
69         loop = asyncio.new_event_loop()
70         asyncio.set_event_loop(loop)
71
72     # execute workflow
73     return loop.run_until_complete(asyncio.to_thread(func_pointer, **kwargs)) or {}
74
75
76 # =====
77 # ⚔ SUPER: THE SUPREME AGENT (ENTERPRISE SOLUTION SUPER AGENT)
78 # =====
79 class AbstractCrewEnterpriseSuperAgent(AbstractSubAgent):
80     """
81     A Class-based Agent representing the Supreme Gatekeeper.
82     It scans Idea, SRS, and Blueprint files for gaps, loopholes, and enterprise compliance.
83     """
84     def __init__(self, agent_id, **kwargs):
85         super().__init__(agent_id=agent_id, **kwargs)
86
87     def file_blueprint_approval(self, prefix):
88         project_name = self.__current_project_name__()
89         prefix = prefix if prefix else "_"
90         return self.__storage_path__(
91             storage_name="storage_sa",
92             file=f"{project_name}/context/{prefix}-csro.{project_name}.global.blueprint.md"
93         ) if project_name else None
94
95     def file_blueprint_diff_analysis(self, prefix):
96         project_name = self.__current_project_name__()
97         prefix = prefix if prefix else "_"
98         return self.__storage_path__(
99             storage_name="storage_sa",
100             file=f"{project_name}/context/{prefix}-diff-analysis.{project_name}
101             }.global.blueprint.md"
102         ) if project_name else None
103
104     def file_ba_approval(self, prefix):
105         project_name = self.__current_project_name__()
106         prefix = prefix if prefix else "_"
107         return self.__storage_path__(
108             storage_name="storage_ba",
109             file=f"{project_name}/{prefix}-csro.requirements.md"
110         ) if project_name else None
111
112     def file_report_ba_approval(self, prefix):
113         project_name = self.__current_project_name__()
114         prefix = prefix if prefix else "_"
115         return self.__storage_path__(
116             storage_name="storage_csro",
117             file=f"{project_name}/{prefix}.chief-solution-report-ba.md"
118         ) if project_name else None
119
120     def file_report_sa_approval(self, prefix):
121         project_name = self.__current_project_name__()
122         prefix = prefix if prefix else "_"
123         return self.__storage_path__(
124             storage_name="storage_csro",
125             file=f"{project_name}/{prefix}.chief-solution-report-sa.md"
126         ) if project_name else None
127
128     def file_report_da_approval(self, prefix):
129         project_name = self.__current_project_name__()
130         prefix = prefix if prefix else "_"
131         return self.__storage_path__(
132             storage_name="storage_csro",
133             file=f"{project_name}/{prefix}.chief-solution-report-diff-analysis.md"
134         ) if project_name else None

```

```

134
135 def file_report_sentinel_approval(self, prefix):
136     project_name = self.__current_project_name__()
137     prefix = prefix if prefix else "_"
138     return self.__storage_path__(
139         storage_name="storage_csro",
140         file=f"{project_name}/{prefix}.chief-solution-report-sentinel.md"
141     ) if project_name else None
142
143 def file_report_kickoff(self, prefix):
144     project_name = self.__current_project_name__()
145     prefix = prefix if prefix else "_"
146     return self.__storage_path__(
147         storage_name="storage_csro",
148         file=f"{project_name}/{prefix}.chief-solution-kickoff.md"
149     ) if project_name else None
150
151 def template_prompt_backstory_sentinel(self):
152     return self.__agents_path__(storage_name="storage_csro_prompts", file=
153         PROMPT_TEMPLATE_BACKSTORY_SOLUTION_SENTINEL)
154
155 def template_prompt_backstory_ba(self):
156     return self.__agents_path__(storage_name="storage_csro_prompts", file=
157         PROMPT_TEMPLATE_BACKSTORY_BA)
158
159 def template_prompt_backstory_sa(self):
160     return self.__agents_path__(storage_name="storage_csro_prompts", file=
161         PROMPT_TEMPLATE_BACKSTORY_SA)
162
163 def template_prompt_backstory_diff_analysis(self):
164     return self.__agents_path__(storage_name="storage_csro_prompts", file=
165         PROMPT_TEMPLATE_BACKSTORY_DIFF_ANALYZER)
166
167 def template_prompt_exptected_sentinel(self):
168     return self.__agents_path__(storage_name="storage_csro_prompts", file=
169         PROMPT_TEMPLATE_EXPECTED_SOLUTION_SENTINEL)
170
171 def template_prompt_expected_ba(self):
172     return self.__agents_path__(storage_name="storage_csro_prompts", file=
173         PROMPT_TEMPLATE_EXPECTED_BA)
174
175 def template_prompt_expected_sa(self):
176     return self.__agents_path__(storage_name="storage_csro_prompts", file=
177         PROMPT_TEMPLATE_EXPECTED_SA)
178
179 def template_prompt_expected_diff_analysis(self):
180     return self.__agents_path__(storage_name="storage_csro_prompts", file=
181         PROMPT_TEMPLATE_EXPECTED_DIFF_ANALYZER)
182
183 def template_prompt_task_sentinel(self):
184     return self.__agents_path__(storage_name="storage_csro_prompts", file=
185         PROMPT_TEMPLATE_TASK_SOLUTION_SENTINEL)
186
187 def template_prompt_task_ba(self):
188     return self.__agents_path__(storage_name="storage_csro_prompts", file=
189         PROMPT_TEMPLATE_TASK_BA)
190
191 def template_prompt_task_sa(self):
192     return self.__agents_path__(storage_name="storage_csro_prompts", file=
193         PROMPT_TEMPLATE_TASK_SA)
194
195 def template_prompt_task_diff_analysis(self):
196     return self.__agents_path__(storage_name="storage_csro_prompts", file=
197         PROMPT_TEMPLATE_TASK_DIFF_ANALYZER)
198
199 def __common_prompt_context__(self):
200     datetime_prompt, datetime_docid = datetime_for_agent()
201     return {
202         "idea_id": self.idea_id,

```

```

191         "language": self.Language,
192         "project_name": self.__current_project_name__() or "-",
193         "project_description": self.__current_project_description__() or "-",
194         "current_timestamp": datetime_prompt,
195         "doc_id": datetime_docid
196     }
197
198 def __pre_initialize__(self):
199     # require idea identity to analyze
200     if not self.project_info:
201         self.logger.critical(f" (1) Invalid idea identity / project name to analyze!")
202         sys.exit(1)
203
204     # check idea file
205     abs_idea_file, phys_idea_file = self.__idea_files__()
206     if not os.path.exists(phys_idea_file):
207         self.logger.critical(f" (4) Not found IDEA file {abs_idea_file}")
208         sys.exit(1)
209     else:
210         self.idea_file = abs_idea_file
211
212     # check requirments file
213     self.ba_file = self.__ba_file__()
214     if not os.path.exists(self.ba_file):
215         self.logger.critical(f" (5) Not found BA file by idea identity / project name '{self.idea_id}'")
216         sys.exit(1)
217
218     # check blueprint file
219     self.blueprint_file = self.__sa_file__()
220     if not os.path.exists(self.blueprint_file):
221         self.logger.critical(f" (6) Not found BLUEPRINT file by idea identity / project name '{self.idea_id}'")
222         sys.exit(1)
223
224 def __create_ai_client__(self):
225     return LLM(
226         model=self.config_model_name(),
227         base_url=self.config_api_endpoint(),
228         api_key=self.__config_api_key__(),
229         provider="openrouter", # force LLM provider
230         temperature=self.agent_temperature()
231     )
232
233 @abstractmethod
234 def __create_llm_agent__(self, **kwargs):
235     pass
236
237 @abstractmethod
238 def __create_agent_task__(self, **kwargs) -> Task:
239     pass
240
241 # @override
242 def initialize(self):
243     # pre-initialize
244     self.__pre_initialize__()
245
246     # initialize super
247     super().initialize()
248     self.language = self.get_kwargs("language") or DEFAULT_CSRO_LANGUAGE
249
250 # @override
251 def agent_log_file(self) -> str:
252     pass
253
254 # @override
255 def system_prompt_template(self) -> str:
256     pass
257

```

```

258     # @override
259     def build_system_prompt(self, **kwargs) -> str:
260         pass
261
262     # @override
263     def user_prompt_template(self) -> str:
264         pass
265
266     # @override
267     def build_user_prompt(self, **kwargs) -> str:
268         pass
269
270     # @override
271     def agent_temperature(self):
272         return 0.1 # require strickly, exact requirements
273
274     # @override
275     def process_communication(self, **kwargs):
276         pass
277
278     # @override
279     def pre_execute(self, **kwargs):
280         pass
281
282
283 # =====
284 # ♂ CLASS 1: THE SUPREME REVIEWER AGENT (ENTERPRISE SOLUTION SENTINEL)
285 # =====
286 class EnterpriseSolutionSentinelAgent(AbstractCrewEnterpriseSuperAgent):
287     """
288     A Class-based Agent representing the Supreme Gatekeeper.
289     It scans Idea, SRS, and Blueprint files for gaps, loopholes, and enterprise compliance.
290     """
291     def __init__(self, **kwargs):
292         super().__init__(agent_id='♂ EnterpriseSolutionSentinelAgent', **kwargs)
293
294     # @override
295     def initialize(self):
296         pass # no need to initialize, just need creating agent/task
297
298     # @override
299     def __create_llm_agent__(self, **kwargs):
300         self.agent = Agent(
301             role="Enterprise Solution Sentinel & Principal / Senior Architecture Gatekeeper",
302             goal="Audit system alignment across Idea, SRS, and Blueprint. Detect loopholes
303                 and enforce structural fixes.",
304             backstory=self.get_kwargs_by_key(key="backstory_sentinel", **kwargs),
305             llm=self.get_kwargs_by_key(key="llm", **kwargs),
306             verbose=True,
307             max_iter=1, # maximum 1 iteration
308             allow_delegation=False # to avoid loop
309         )
310         return self.agent
311
312     # @override
313     def __create_agent_task__(self, **kwargs) -> Task:
314         """
315         Generates the core evaluation task with injectible document payloads.
316         """
317         return Task(
318             description=self.get_kwargs_by_key(key="task_sentinel", **kwargs),
319             expected_output=self.get_kwargs_by_key(key="expected_sentinel", **kwargs),
320             agent=self.agent
321         )
322
323     # @override
324     def __create_ai_client__(self):
325         pass

```

```

326     # @override
327     def __ai_execute__(self, **kwargs):
328         pass
329
330     # =====
331     # □ CLASS 2: BUSINESS ANALYST AGENT
332     # =====
333     class EnterpriseBusinessAnalystAgent(AbstractCrewEnterpriseSuperAgent):
334         """
335         A Class-based Agent responsible for authoring and revising the SRS.
336         """
337         def __init__(self, **kwargs):
338             super().__init__(agent_id='□ EnterpriseBusinessAnalystAgent', **kwargs)
339
340         # @override
341         def initialize(self):
342             pass # no need to initialize, just need creating agent/task
343
344         # @override
345         def __create_llm_agent__(self, **kwargs):
346             self.agent = Agent(
347                 role="Enterprise Business Analyst",
348                 goal="Author and overhaul software requirements specifications ensuring absolute
349                     alignment with product ideas.",
350                 backstory=self.get_kwargs_by_key(key="backstory_ba", **kwargs),
351                 llm=self.get_kwargs_by_key(key="llm", **kwargs),
352                 verbose=True,
353                 max_iter=1, # maximum 1 iteration
354                 allow_delegation=False # to avoid loop
355             )
356             return self.agent
357
358         # @override
359         def __create_agent_task__(self, **kwargs) -> Task:
360             return Task(
361                 description=self.get_kwargs_by_key(key="task_ba", **kwargs),
362                 expected_output=self.get_kwargs_by_key(key="expected_ba", **kwargs),
363                 agent=self.agent,
364                 context=self.get_kwargs_by_key(key="context_tasks_ba", **kwargs)
365             )
366
367         # @override
368         def __create_ai_client__(self):
369             pass
370
371         # @override
372         def __ai_execute__(self, **kwargs):
373             pass
374
375     # =====
376     # □ CLASS 3: SYSTEM ARCHITECT AGENT
377     # =====
378     class EnterpriseSystemArchitectAgent(AbstractCrewEnterpriseSuperAgent):
379         """
380         A Class-based Agent responsible for structural and infrastructural Blueprints.
381         """
382         def __init__(self, **kwargs):
383             super().__init__(agent_id='□ EnterpriseSystemArchitectAgent', **kwargs)
384
385         # @override
386         def initialize(self):
387             pass # no need to initialize, just need creating agent/task
388
389         # @override
390         def __create_llm_agent__(self, **kwargs):
391             self.agent = Agent(
392                 role="Enterprise System Architect",
393                 goal="Architect and refactor system blueprint infrastructures to match software

```

```

394         specifications.",
395         backstory=self.get_kwarg_by_key(key="backstory_sa", **kwargs),
396         llm=self.get_kwarg_by_key(key="llm", **kwargs),
397         verbose=False,
398         max_iter=1,                # maximum 1 iteration
399         allow_delegation=False     # to avoid loop
400     )
401     return self.agent
402
403 # @override
404 def __create_agent_task__(self, **kwargs) -> Task:
405     return Task(
406         description=self.get_kwarg_by_key(key="task_sa", **kwargs),
407         expected_output=self.get_kwarg_by_key(key="expected_sa", **kwargs),
408         agent=self.agent,
409         context=self.get_kwarg_by_key(key="context_tasks_sa", **kwargs)
410     )
411
412 # @override
413 def __create_ai_client__(self):
414     pass
415
416 # @override
417 def __ai_execute__(self, **kwargs):
418     pass
419
420 # =====
421 # ❧ CLASS 4: THE SUPREME SYSTEM ARCHITECTURE WORKFLOW AGENT (ENTERPRISE WORKFLOW)
422 # =====
423 class AbstractCrewEnterpriseWorkflowAgent(AbstractCrewEnterpriseSuperAgent):
424     """
425     A Class-based Agent representing the Supreme Gatekeeper.
426     It scans Idea, SRS, and Blueprint files for gaps, loopholes, and enterprise compliance.
427     """
428     def __init__(self, agent_id, **kwargs):
429         super().__init__(agent_id=agent_id, **kwargs)
430
431     @abstractmethod
432     def build_prompts(self, **kwargs):
433         pass
434
435     # @override
436     def agent_log_file(self) -> str:
437         return self.__output_storage_path__(storage_name="output_csro", file=CSRO_LOG_FILE)
438
439     @abstractmethod
440     def __build_arguments_for_communicating__(self, **kwargs):
441         pass
442
443     def __reset_crew_event_bus_to_rotate_model__(self):
444         try:
445             # 1. reset events counter of CrewAI
446             reset_emission_counter()
447
448             # 2. for ContextVar contains Event ID Stack to empty tuple ()
449             _event_id_stack.set(())
450             _event_context_config.set(EventContextConfig())
451
452             # 3. reset stuck sync/async handlers / events
453             if hasattr(crewai_event_bus, '_sync_handlers'):
454                 crewai_event_bus._sync_handlers.clear()
455             if hasattr(crewai_event_bus, '_async_handlers'):
456                 crewai_event_bus._async_handlers.clear()
457             if hasattr(crewai_event_bus, '_event_scopes'):
458                 crewai_event_bus._event_scopes = []
459             if hasattr(crewai_event_bus, '_events'):
460                 crewai_event_bus._events = {}
461             print(f"[ {self.agent_id} Agent | CLEAN ] Reset present Event Stack to rotate

```



```

        new model.")
        return True
    except Exception as e:
        print(f"[ {self.agent_id} Agent | ERROR ] Could not reset Event Bus: {str(e)}")
        return False

# @override
def __rotate_next_model__(self):
    # require clear event stack to avoid events stuck before rotating new model
    if self.__reset_crew_event_bus_to_rotate_model__():
        return super().__rotate_next_model__()
    return False

# @override
def __communicate_ai__(self, **kwargs):
    # build arguments
    built_kwargs = self.__build_arguments_for_communicating__(**kwargs) or {}

    # create CrewAI
    crew_ai = Crew(
        agents=self.get_kwargs_by_key(key="agents", **built_kwargs),
        tasks=self.get_kwargs_by_key(key="tasks", **built_kwargs),
        process=Process.sequential
    )

    # kick-off CrewAI
    return crew_ai.kickoff()

# @override
def __ai_execute__(self, **kwargs):
    # build prompts first
    kwargs = self.build_prompts(**kwargs)

    # execute
    return super().__ai_execute__(**kwargs)

# @override
def __do_execute__(self, **kwargs):
    # execute
    kwargs = super().__do_execute__(**kwargs)

    # success, due to not reach exception from super function, do delete Log if neccessary
    delete_file(file=self.agent_log_file())

    # return result
    return kwargs

# =====
# ♂ CLASS 5: THE SUPREME SYSTEM ARCHITECTURE WORKFLOW AGENT (ENTERPRISE WORKFLOW)
# =====
class CrewEnterpriseSolutionWorkflowAgent(AbstractCrewEnterpriseWorkflowAgent):
    """
    A Class-based Agent representing the Supreme Gatekeeper.
    It scans Idea, SRS, and Blueprint files for gaps, loopholes, and enterprise compliance.
    """
    def __init__(self, **kwargs):
        super().__init__(agent_id='♂ EnterpriseSolutionWorkflowReviewerAgent', **kwargs)

    # @override
    def agent_log_file(self) -> str:
        return self.__output_storage_path__(storage_name="output_csro", file=CSRO_LOG_FILE)

    # @override
    def build_prompts(self, **kwargs):
        return {
            **kwargs,
            "backstory_sentinel": render_kwargs_prompt(self.template_prompt_backstory_sentinel
            ()), **kwargs),

```



```

529         "backstory_ba": render_kwargs_prompt(self.template_prompt_backstory_ba(), **kwargs
530         ),
531         "backstory_sa": render_kwargs_prompt(self.template_prompt_backstory_sa(), **kwargs
532         ),
533         "expected_sentinel": render_kwargs_prompt(self.template_prompt_expected_sentinel
534         ()), **kwargs),
535         "expected_ba": render_kwargs_prompt(self.template_prompt_expected_ba(), **kwargs),
536         "expected_sa": render_kwargs_prompt(self.template_prompt_expected_sa(), **kwargs),
537
538         "task_sentinel": render_kwargs_prompt(self.template_prompt_task_sentinel(), **
539         kwargs),
540         "task_ba": render_kwargs_prompt(self.template_prompt_task_ba(), **kwargs),
541         "task_sa": render_kwargs_prompt(self.template_prompt_task_sa(), **kwargs),
542     }
543
544     # @override
545     def __create_llm_agent__(self, **kwargs):
546         # re-initialialize agent classes to release memory
547         self.agent_solution_sentinel = EnterpriseSolutionSentinelAgent(**kwargs)
548         self.agent_business_analyst = EnterpriseBusinessAnalystAgent(**kwargs)
549         self.agent_system_architect = EnterpriseSystemArchitectAgent(**kwargs)
550
551         # create internal agents
552         self.agent_solution_sentinel.__create_llm_agent__(**kwargs)
553         self.agent_business_analyst.__create_llm_agent__(**kwargs)
554         self.agent_system_architect.__create_llm_agent__(**kwargs)
555         return None
556
557     # @override
558     def __create_agent_task__(self, **kwargs) -> Task:
559         # solution sentinel task
560         self.task_solution_sentinel = self.agent_solution_sentinel.__create_agent_task__(**
561         kwargs)
562
563         # business analyst task
564         kwargs = {
565             **kwargs,
566             "context_tasks_ba": [ self.task_solution_sentinel ]
567         }
568         self.task_business_analyst = self.agent_business_analyst.__create_agent_task__(**
569         kwargs)
570
571         # system architect task
572         kwargs = {
573             **kwargs,
574             "context_tasks_sa": [ self.task_solution_sentinel, self.task_business_analyst ]
575         }
576         self.task_system_architect = self.agent_system_architect.__create_agent_task__(**
577         kwargs)
578         return None
579
580     # @override
581     def pre_execute(self, **kwargs):
582         # read idea file
583         _, raw_idea_content = self.__read_idea_or_requirements__(ignore_not_found=True)
584
585         # no idea also no requirements
586         if not raw_idea_content:
587             self.logger.critical(f"❑ Not found IDEA / Requirements file to process")
588             sys.exit(1)
589
590         # read BA file
591         raw_ba_content = self.__read_srs__(ignore_not_found=False)
592
593         # read Blueprint file
594         raw_blueprint_content = self.__read_blueprint__(ignore_not_found=False)
595
596         # return merged new values

```

```

591         return {
592             **kwargs,
593             **self.__common_prompt_context__(),
594             "raw_idea_content": raw_idea_content,
595             "raw_srs_content": raw_ba_content,
596             "raw_blueprint_content": raw_blueprint_content
597         }
598
599     # @override
600     def __build_arguments_for_communicating__(self, **kwargs):
601         # initialize LLM model, belongs to rotating models
602         built_kwargs = {
603             **kwargs,
604             "llm": self.client
605         }
606
607         # create internal LLM agents
608         self.__create_llm_agent__(**built_kwargs)
609
610         # create task
611         self.__create_agent_task__(**built_kwargs)
612
613         # initial crew with internal agent, task
614         agents = [
615             self.agent_solution_sentinel.agent,
616             self.agent_business_analyst.agent,
617             self.agent_system_architect.agent,
618         ]
619         tasks = [
620             self.task_solution_sentinel,
621             self.task_business_analyst,
622             self.task_system_architect,
623         ]
624         return {
625             **built_kwargs,
626             "agents": agents,
627             "tasks": tasks
628         }
629
630     # @override
631     def __parse_ai_response__(self, response):
632         # □ Extract output of tasks
633         raw_sentinel_response = None
634         raw_ba_response = None
635         raw_sa_response = None
636         try:
637             # Task 1 (Sentinel) response - Audit Report
638             raw_sentinel_response = self.task_solution_sentinel.output.raw
639
640             # Task 2 (Business Analyst) response
641             raw_ba_response = self.task_business_analyst.output.raw
642
643             # Task 3 (System Architect) response - fixed blueprint
644             raw_sa_response = self.task_system_architect.output.raw
645         except Exception as e:
646             self.logger.error(f"□ Could extract task output: {str(e)}")
647
648         # parsed responses
649         return {
650             "kickoff": response,
651             "report_sentinel": raw_sentinel_response,
652             "report_ba": raw_ba_response,
653             "report_sa": raw_sa_response
654         }
655
656     # @override
657     def process_communication(self, **kwargs):
658         response_data = self.get_kwargs_by_key(key="clean_response", **kwargs)
659         if not response_data or not isinstance(response_data, dict):

```

```

660         raise RuntimeError(f" (7) Invalid AI raw response.")
661
662     # project info
663     timestamp = self.get_kwarg_by_key(key="current_timestamp_2", **kwargs)
664
665     # export storage audit report
666     if "kickoff" in response_data and response_data.get("kickoff"):
667         write_file(file=self.file_report_kickoff(prefix=timestamp), data=response_data.get(
668             ("kickoff")))
669     else:
670         self.logger.warning(f" No any kickoff report!")
671
672     # export task sentinel response
673     if "report_sentinel" in response_data and response_data.get("report_sentinel"):
674         write_file(file=self.file_report_sentinel_approval(prefix=timestamp), data=
675             response_data.get("report_sentinel"))
676     else:
677         self.logger.warning(f" No any sentinel report!")
678
679     # export task BA response
680     if "report_ba" in response_data and response_data.get("report_ba"):
681         response_ba = response_data.get("report_ba")
682         write_file(file=self.file_report_ba_approval(prefix=timestamp), data=response_ba)
683         write_file(file=self.file_ba_approval(prefix=timestamp), data=response_ba)
684     else:
685         self.logger.warning(f" No any business-analysis report!")
686
687     # export task SA response
688     if "report_sa" in response_data and response_data.get("report_sa"):
689         response_sa = response_data.get("report_sa")
690         write_file(file=self.file_report_sa_approval(prefix=timestamp), data=response_sa)
691         write_file(file=self.file_blueprint_approval(prefix=timestamp), data=response_sa)
692     else:
693         self.logger.warning(f" No any system-architect report!")
694
695     # export raw response if necessary as Log tracing
696     raw_response = self.get_kwarg_by_key(key="raw_response", **kwargs)
697     if raw_response:
698         write_file(
699             file=self.__output_storage_path__(storage_name="output_csro", file=
700                 CSRO_RAW_FILE),
701             data=raw_response
702         )
703
704     return { **kwargs }
705
706 # =====
707 # CLASS 6: SYSTEM ARCHITECT DIFF ANALYZER AGENT
708 # =====
709 class CrewEnterpriseBlueprintDiffAnalyzerAgent(AbstractCrewEnterpriseWorkflowAgent):
710     """
711     A Class-based Agent representing the Supreme Gatekeeper.
712     It scans Idea, SRS, and Blueprint files for gaps, loopholes, and enterprise compliance.
713     """
714     def __init__(self, **kwargs):
715         super().__init__(agent_id=' EnterpriseBlueprintDiffAnalyzerAgent', **kwargs)
716
717     # @override
718     def build_prompts(self, **kwargs):
719         return {
720             **kwargs,
721             "backstory_da": render_kwarg_prompt(self.template_prompt_backstory_diff_analysis(
722                 (), **kwargs),
723             "task_da": render_kwarg_prompt(self.template_prompt_task_diff_analysis(), **
724                 kwargs),
725             "expected_da": render_kwarg_prompt(self.template_prompt_expected_diff_analysis(),
726                 **kwargs)
727         }

```

```

723
724 # @override
725 def __create_llm_agent__(self, **kwargs):
726     self.agent = Agent(
727         role="Principal Enterprise Systems Auditor",
728         goal="Execute independent triple-check architectural audits on system blueprints."
729         ,
730         backstory=self.get_kwargs_by_key(key="backstory_da", **kwargs),
731         llm=self.get_kwargs_by_key(key="llm", **kwargs),
732         verbose=False,
733         max_iter=1, # maximum 1 iteration
734         allow_delegation=False # to avoid loop
735     )
736     return self.agent
737
738 # @override
739 def __create_agent_task__(self, **kwargs) -> Task:
740     return Task(
741         description=self.get_kwargs_by_key(key="task_da", **kwargs),
742         expected_output=self.get_kwargs_by_key(key="expected_da", **kwargs),
743         agent=self.agent
744     )
745
746 # @override
747 def agent_log_file(self) -> str:
748     return self.__output_storage_path__(storage_name="output_csro", file=CSRO_LOG_DA_FILE)
749
750 # @override
751 def pre_execute(self, **kwargs):
752     # read Blueprint file
753     raw_blueprint_content = self.__read_blueprint__()
754
755     # return merged new values
756     return {
757         **kwargs,
758         **self.__common_prompt_context__(),
759         "raw_blueprint_content": raw_blueprint_content
760     }
761
762 def __build_arguments_for_communicating__(self, **kwargs):
763     # initialize LLM model, belongs to rotating models
764     built_kwargs = {
765         **kwargs,
766         "llm": self.client
767     }
768
769     # initial crew with internal agent, task
770     agents = [ self.__create_llm_agent__(**built_kwargs) ]
771     tasks = [ self.__create_agent_task__(**built_kwargs) ]
772     return {
773         **built_kwargs,
774         "agents": agents,
775         "tasks": tasks
776     }
777
778 # @override
779 def process_communication(self, **kwargs):
780     response_data = self.get_kwargs_by_key(key="clean_response", **kwargs)
781     if not response_data or not isinstance(response_data, dict):
782         raise RuntimeError(f"[ {self.agent_id} Agent | CRITICAL ERROR ] (7) Invalid AI raw response.")
783
784     # project info
785     timestamp = self.get_kwargs_by_key(key="current_timestamp_2", **kwargs)
786
787     # export task DA response
788     write_file(file=self.file_report_da_approval(prefix=timestamp), data=response_data)
789     write_file(file=self.file_blueprint_diff_analysis(prefix=timestamp), data=
response_data)

```

```

789
790     # export raw response if necessary as Log tracing
791     raw_response = self.get_kwargs_by_key(key="raw_response", **kwargs)
792     if raw_response:
793         write_file(
794             file=self.__output_storage_path__(storage_name="output_csro", file=
795                 CSRO_DA_FILE),
796             data=raw_response
797         )
798     return { **kwargs }
799
800
801 # =====
802 # 🐧 FINAL: THE SUPREME WORKFLOW AGENT (ENTERPRISE WORKFLOW)
803 # =====
804 class CrewEnterpriseGovernanceFlow(Flow):
805     """
806     An asynchronous, stateful event-driven workflow execution layer.
807     Utilizes native CrewAI Flow hooks (@start, @listen) to eliminate
808     Pydantic re-validation failure loops and preserve clean memory handoffs.
809     """
810
811     def __init__(self, **kwargs):
812         super().__init__()
813         # Inject pre-initialized custom wrappers containing the actual CrewAI Agents
814         self.kwargs = kwargs or {}
815         self.logger = get_logger("CrewEnterpriseGovernanceFlow")
816         self.agent_solution_review = CrewEnterpriseSolutionWorkflowAgent(**self.kwargs)
817         self.agent_diff_analyzer = CrewEnterpriseBlueprintDiffAnalyzerAgent(**self.kwargs)
818
819     @start()
820     def execute_solution_review(self):
821         """
822         STAGE 1: Kicks off the linear 3-Agent generation sub-crew (Sentinel -> BA -> SA).
823         Returns the raw modified blueprint string fetched directly from SA's RAM cache.
824         """
825         self.logger.info("[ 🐧 SOLUTION ARCHITECT REVIEW ] Enterprise Solution Architecture
826             Review CSRO...")
827         return __execute_function_until_complete__(func_pointer=self.agent_solution_review.
828             execute) or {}
829
830     @listen(execute_solution_review)
831     def execute_blueprint_diff_analysis(self, solution_architect_review_result):
832         """
833         STAGE 2: Explicitly triggered via native event hooks once STAGE 1 returns
834         successfully.
835         Ingests the fixed blueprint payload into your explicit custom template keyword.
836         """
837         self.logger.info("[ 🐧 BLUEPRINT CHANGES ANALYSIS ] Analyze new changes of Solution
838             Architecture Report...")
839         kwargs = {
840             **solution_architect_review_result,
841             "raw_csro_blueprint_content": solution_architect_review_result.get("report_sa",
842                 None)
843         }
844         return __execute_function_until_complete__(func_pointer=self.agent_diff_analyzer.
845             execute, **kwargs) or {}
846
847 if __name__ == "__main__":
848     def add_known_arguments(parser):
849         parser.add_argument("--idea", type=str, help="Idea Identity for searching")
850         parser.add_argument("--language", type=str, help="Translate SRS to language. Ex:
851             Vietnamese, English, etc.")
852
853     args, unknown_args = parse_args(
854         description="🐧 CrewEnterpriseGovernanceFlow",
855         parser_callback=add_known_arguments

```

```
850 )
851
852 # force litellm stop collecting old exceptions to metadata
853 litellm.suppress_helper_warnings = True
854 litellm.drop_params = True
855
856 # initializ workflow agent
857 enterprise_workflow_agent = CrewEnterpriseGovernanceFlow(
858     idea=args.idea,
859     language=args.Language,
860     **unknown_args
861 )
862
863 # execute workflow
864 __execute_function_until_complete__(enterprise_workflow_agent.kickoff)
865
866
867
```